

AMEN BANK

Direction Centrale du système d'information

SUJET DE STAGE

Détection d'anomalies par Deep Learning sur les opérations de caisse en agence bancaire

Retrait · Versement · Virement · Remise de Chèques

1. Informations générales

Établissement	Amen Bank
Département	Département des projets agence
Langage principal	Python / Java
Domaine	Deep Learning — Intelligence Artificielle Appliquée

2. Contexte et problématique

En agence bancaire, les opérations de caisse — retrait espèces, versement espèces, virement bancaire et remise de chèques — génèrent quotidiennement un volume important de transactions. Ces flux exposent l'établissement à plusieurs catégories de risques :

- Fraude active — *faux chèques, virements détournés, retraits usurpés par un tiers*
- Blanchiment d'argent — *fractionnement de versements (smurfing), dépôts atypiques — risque LAB/FT*
- Erreurs opérationnelles — *doublons de saisie, mauvais comptes, montants incorrects*
- Comportements inhabituels — *rupture soudaine du pattern habituel d'un client*

Aujourd'hui, la détection de ces anomalies repose principalement sur des règles métier manuelles et des contrôles a posteriori. L'objectif de ce stage est de concevoir, en Python, un système intelligent basé sur le Deep Learning capable d'identifier en quasi-temps-réel toute opération suspecte parmi ces quatre types, afin d'assister le superviseur d'agence dans sa prise de décision.

3. Périmètre fonctionnel — Opérations couvertes

Opération	Anomalies et comportements suspects ciblés
Retrait espèces	Montant anormalement élevé, fréquence excessive en 24h, retrait à heure inhabituelle, retraits multiples GAB
Versement espèces	Fractionnement de dépôts (smurfing), montants ronds suspects (1000, 2000 DT), cumul sur 48h anormal
Virement bancaire	Bénéficiaire inconnu, virement en doublon, montant hors norme, virement hors horaires ouvrables
Remise de chèques	Chèque sans provision, émetteur jamais vu, montant incohérent avec l'historique client, endossement à un tiers

4. Objectifs pédagogiques

À l'issue de ce stage, le stagiaire aura acquis les compétences suivantes :

- Compréhension du métier bancaire — *workflow des opérations de caisse, règles LAB/FT, seuils réglementaires Banque Centrale de Tunisie*
- Maîtrise du Deep Learning appliqué — *Autoencoder pour la détection non supervisée, LSTM pour l'analyse de séquences temporelles*
- Feature engineering sur données financières — *construction de variables pertinentes à partir de données transactionnelles*
- Évaluation rigoureuse de modèles — *AUC-ROC, F1-score, gestion du déséquilibre de classes*
- Explainabilité des modèles IA — *intégration de SHAP pour justifier chaque alerte en langage naturel*
- Développement d'une interface utilisateur — *dashboard Streamlit destiné aux superviseurs d'agence*

5. Approche technique

5.1 Génération des données

En l'absence de données réelles disponibles, le stagiaire générera un dataset synthétique réaliste à l'aide de bibliothèques Python spécialisées (Faker, SDV). Ce dataset simulera les quatre types d'opérations avec injection contrôlée d'anomalies représentatives des cas de fraude et de comportements atypiques.

5.2 Modèle 1 — Autoencoder (détection non supervisée)

L'Autoencoder est un réseau de neurones entraîné uniquement sur des opérations normales. Il apprend à compresser puis reconstruire les données. Une opération présentant une erreur de reconstruction élevée (c'est-à-dire que le modèle ne sait pas bien la reconstruire) est considérée comme anormale. C'est la technique la plus adaptée au contexte bancaire où les anomalies sont rares et souvent non étiquetées.

5.3 Modèle 2 — LSTM (analyse séquentielle)

Le LSTM (Long Short-Term Memory) traite l'historique des opérations d'un client comme une séquence temporelle. Il modélise le comportement habituel du client dans le temps et signale toute rupture inhabituelle dans cette séquence. Cette approche est particulièrement efficace pour détecter les changements de comportement progressifs (ex. : augmentation soudaine du volume de virements).

5.4 Explainabilité avec SHAP

Pour chaque alerte générée, le module SHAP (SHapley Additive exPlanations) produira une explication lisible indiquant quelles variables ont le plus contribué au score d'anomalie. Exemple : « Ce versement est suspect car le montant est 4x la moyenne du client, c'est le 3e dépôt en 48h, et le montant est un multiple rond de 1000 DT. »

5.5 Dashboard superviseur (Streamlit)

L'application finale comprendra les écrans suivants :

- Tableau des alertes — *liste des opérations suspectes avec score de suspicion et type d'anomalie*
- Détail d'une alerte — *explication SHAP en langage naturel, graphique des features déterminantes*
- KPIs agence — *nombre d'alertes par type d'opération, par agence, par jour*
- Mode simulation — *saisir manuellement une opération et obtenir son score instantanément*
- Validation superviseur — *confirmer ou rejeter une alerte pour alimenter un feedback loop*

6. Plan de travail — 8 semaines

Semaine	Thème	Travaux à réaliser
Sem. 1	Analyse métier & cadrage	Étudier le workflow des 4 opérations (retrait, versement, virement, chèque). Recenser les règles LAB/FT et seuils réglementaires. Revue bibliographique : Autoencoder, LSTM, Isolation Forest. Définir le schéma de données et les cas d'anomalies.
Sem. 2	Données & exploration	Générer un dataset synthétique avec Faker/SDV. Injecter des anomalies réalistes (smurfing, doublons, montants hors norme). Réaliser l'analyse exploratoire (distributions, corrélations, déséquilibre de classes). Feature engineering par type d'opération.
Sem. 3	Modèle 1 — Autoencoder	Implémenter un Autoencoder avec PyTorch. Entraîner le modèle sur les opérations normales uniquement. Calculer le score d'anomalie via l'erreur de reconstruction. Calibrer le seuil d'alerte (percentile 95/99).
Sem. 4	Modèle 2 — LSTM	Implémenter un réseau LSTM pour l'analyse séquentielle. Modéliser l'historique des opérations client comme une séquence temporelle. Détecter les ruptures de comportement dans les séquences. Gérer les séquences de longueur variable (padding, masking).
Sem. 5	Évaluation & comparaison	Calculer les métriques : AUC-ROC, F1-score, Recall, Precision. Comparer les deux modèles Deep Learning avec les baselines classiques (Isolation Forest, LOF). Calibrer les seuils pour limiter la fatigue d'alerte. Produire les matrices de confusion par type d'opération.
Sem. 6	Explainabilité SHAP	Intégrer SHAP pour expliquer chaque alerte en langage naturel. Identifier les features les plus déterminantes par opération. Générer des messages automatiques lisibles pour le superviseur (ex. : 'Ce virement est suspect car le bénéficiaire est inconnu et le montant est 3x la moyenne').
Sem. 7	Dashboard superviseur	Développer une application Streamlit avec : liste des alertes en temps réel, score de suspicion + explication SHAP, KPIs par type d'opération et par agence, mode simulation pour tester une opération, validation superviseur (confirmer ou rejeter une alerte).
Sem. 8	Finalisation & soutenance	Nettoyer et documenter le code (docstrings, README). Rédiger le rapport de stage (30–40 pages). Préparer la démonstration live. Soutenance finale avec présentation des résultats.

7. Stack technique

Catégorie	Outils / Bibliothèques
Deep Learning	PyTorch — Autoencoder, LSTM
Données synthétiques	Faker, SDV (Synthetic Data Vault)
Data & Preprocessing	Pandas, NumPy, Scikit-learn
Explainabilité	SHAP
Dashboard	Streamlit, Plotly
Visualisation	Matplotlib, Seaborn
Notebooks	Jupyter Notebook
Suivi expériences	MLflow (optionnel)

8. Livrables attendus

Livrable	Format	Semaine
Document de cadrage + glossaire métier	PDF / Word	Sem. 1
Dataset synthétique annoté + Notebook EDA	.csv + .ipynb	Sem. 2
Modèle Autoencoder entraîné	.pt (PyTorch)	Sem. 3
Modèle LSTM entraîné	.pt (PyTorch)	Sem. 4
Rapport comparatif des modèles	PDF / Notebook	Sem. 5
Module d'explainabilité SHAP	.py	Sem. 6
Dashboard superviseur fonctionnel	Application Streamlit	Sem. 7
Code source documenté	.py	Sem. 8